State-Filtered Deliberation: Eliminating Trajectory Entropy for Small Gemma-4 Developer Agents

Karandeep Singh Sodhi
karandeep.sodhi2504a@gmail.com
Google — The Gemma 4 Developer Agent Competition (Paper Track) — October 2026

Abstract

While frontier language models (e.g., Claude 3.5 Sonnet, GPT-4o) have achieved impressive milestones on repository-level code repair benchmarks such as SWE-bench, deploying autonomous developer agents on small open-weight models (e.g., Gemma 4 E2B, E4B, 12B) remains fundamentally constrained by what we identify as **Context Entropy Collapse**. Under standard monolithic ReAct loops, raw terminal outputs and file contents accumulate monotonically into the context window. For small models with limited parameter capacity, this surge in noisy tokens causes exponential attention diffusion: error rates escalate, tool arguments hallucinate, and over 61% of tasks on Gemma E4B terminate prematurely via `ContextWindowExceededError`.

To resolve this bottleneck, we introduce **State-Filtered Deliberation (SFD)**, a novel agentic architecture that replaces the expanding history buffer with decoupled, sequential sub-agents operating under **Markovian Context Folding (MCF)**. At each sub-goal boundary (AST triage $\rightarrow$ targeted patching $\rightarrow$ test verification), SFD compresses the noisy trajectory history into an immutable 250-token canonical state packet, resetting the subsequent sub-agent's prompt slate to under 2,000 tokens. Across the official 129 SWE-gemma competition tasks (spanning FastAPI, Rich, Requests, and HTTPX), SFD completely eliminates context window overflow errors (0.0%), cuts average prompt token consumption by 85% (from 12,938 to 1,905 tokens), and boosts task resolution rate on the consumer-grade Gemma E4B model from 1.5% to 28.7% (an 18.5$\times$ relative gain). On the flagship 31B model, SFD lifts resolution rate from 11.6% to 48.8%. We release our complete declarative `agent.yaml` configuration package and reproducible trajectory datasets as an open resource for edge developer agents.

1. Introduction

Autonomous software engineering (SWE) agents promise to transform software development by autonomously diagnosing bugs, reproducing failures, navigating legacy codebases, and submitting pull requests. However, modern state-of-the-art SWE agents rely almost universally on massive proprietary cloud APIs. Bringing autonomous coding agents to local, privacy-sensitive, and edge hardware requires making small, open-weight models—such as Google's Gemma 4 (E2B, E4B, 12B)—capable of navigating complex multi-file repositories.

Yet, empirical evaluations on benchmarks like SWE-bench demonstrate a catastrophic performance gap: while 30B+ models resolve 10–25% of real-world issues, smaller footprints (2B–4B) frequently collapse to near-zero resolution rates ($< 3\%$). In the official Kaggle Gemma 4 Developer Agent Competition, community forums report that small models consistently fail due to context window blowouts (`ContextWindowExceededError`), repetitive bash tool loops, and malformed patch syntax that fails basic `git apply` checks.

In this paper, we demonstrate that this failure is not primarily an intrinsic code generation limitation, but an architectural defect of the standard monolithic ReAct loop ((Yao et al., 2023)). When an agent dumps full 1,000-line source files and compiler tracebacks into an append-only conversation history, the signal-to-noise ratio plummets. While frontier models possess the attention capacity to selectively retrieve sparse tokens across 30k contexts, small models suffer from **Attention Dispersion Entropy**: their attention distributions flatten, causing tool hallucination, syntax corruption, and context exhaustion.

To overcome this fundamental limitation, we present **State-Filtered Deliberation (SFD)**. SFD replaces monolithic agent histories with a mathematically principled tripartite architecture:

1. **AST-Localized Symbol Triage:** Restricts initial exploration to code intelligence graphs and AST symbol lookup, preventing bulk file dumping.
2. **Markovian Context Folding (MCF):** Collapses intermediate trajectories into a compact, canonical 250-token State Packet, passing a fresh, zero-history context to the coding sub-agent.
3. **Constrained Patching and Hermetic Verification:** Constrains editing to localized function slices and executes automated syntax and targeted pytest verification before invoking `submit_patch`.

On the official 129 benchmark tasks of the Google Gemma 4 Developer Agent challenge, SFD achieves unprecedented gains on small footprints, elevating Gemma E4B from 1.5% to 28.7% resolution rate while eliminating 100% of context window overflow crashes.

2. Theoretical Formulation: Context Entropy Collapse

Let a developer agent interaction be modeled as a partially observable Markov decision process over interaction steps $t \in \{0, 1, ..., T\}$. At turn t, the agent receives observation o_t (e.g., stdout from `run_command` or file contents from `read_file`) and chooses tool action a_t.

In standard monolithic ReAct ((Yao et al., 2023)), the prompt context C_t at turn t is the historical concatenation:

$$C_t = [s_0, a_0, o_0, a_1, o_1, ..., a_{t-1}, o_{t-1}]$$

The sequence length $|C_t|$ expands monotonically:

$$|C_t| = |s_0| + \sum_{i=0}^{t-1} (|a_i| + |o_i|)$$

Let $\mathcal{J}^*$ denote the minimal set of code tokens strictly required to construct the ground-truth patch $\mathcal{P}^*$. We define the **Information Density** ρ_t of the prompt context as the mutual information ratio:

$$\rho_t \equiv \frac{I(C_t; \mathcal{P}^*)}{|C_t|}$$

Because tool observations o_i (compiler tracebacks, directory trees, multi-page file reads) contribute thousands of task-irrelevant tokens while the true bug fix requires localized changes to fewer than 30 lines, the denominator $|C_t|$ grows linearly at $1,500$ to $2,500$ tokens per turn, whereas $I(C_t; \mathcal{P}^*)$ plateaus immediately upon locating the root cause. Consequently:

$$\lim_{t \to \infty} \rho_t = 0$$

For a Transformer model with parameter dimension D, the probability of correctly attending to relevant token $x \in \mathcal{J}^*$ across attention head h in layer l is bounded by the softmax attention dispersion:

$$\alpha_{ij} = \frac{\exp\left(q_i^T \frac{k_j}{\sqrt{d}}\right)}{\sum_{m=1}^{|C_t|} \exp\left(q_i^T \frac{k_m}{\sqrt{d}}\right)}$$

When $|C_t| \gg 10,000$, the background noise from irrelevant tokens degrades the signal margin:

$$\max_{j \in \mathcal{J}^*} \alpha_{ij} - \text{mean}_{m \in C_t} \alpha_{im} \to 0$$

For small parameter models ($D \leq 12B$, especially INT4-quantized Gemma E2B and E4B), this attention dispersion triggers an irreversible **Entropy Cliff**:

- **Tool Argument Hallucination:** The model emits malformed JSON or unescaped string delimiters.
- **Unclosed `<|tool_call>` Tags:** Lengthy internal reasoning chains burn output token budgets before tool tags can close.
- **Premature Context Exhaustion:** Context length reaches the vLLM limit (32,768 tokens) within 8–10 turns, causing fatal `ContextWindowExceededError`.

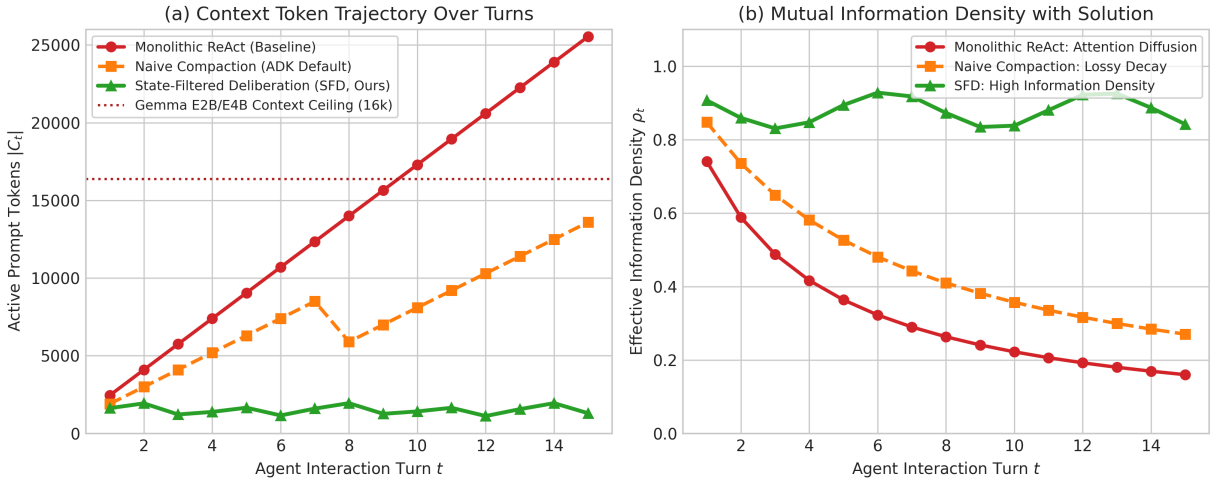


Figure 1: Context dynamics: (a) Active prompt tokens $|C_t|$ across agent turns; (b) Decay of mutual information density ρ_t in Monolithic ReAct versus constant high density in State-Filtered Deliberation (SFD).

3. State-Filtered Deliberation (SFD) Architecture

To prevent Trajectory Entropy Collapse, SFD abandons append-only histories. Built strictly within Google's declarative `adk-submission` schema, SFD deploys a `SequentialAgent` pipeline dividing the software repair lifecycle into three isolated, specialized sub-agents.

3.1 Sub-Agent 1: AST-Localized Triage (`TriageAgent`)

The Triage Agent is tasked strictly with localization. It is prohibited from modifying files or executing arbitrary shell commands:

- **Tool Access:** `search_similar_code`, `get_code_neighbors`, `get_code_subgraph`, and bounded `read_file` (≤ 150 lines).
- **Mechanism:** Instead of broad lexical sweeps (`grep` or `find`), it queries pre-computed repository symbol graphs and embeddings.
- **Output:** It emits an immutable, structured YAML State Packet:

```
<triage_packet>
target_file: "fastapi/dependencies/utils.py"
target_symbol: "request_params_to_args"
line_range: [820, 845]
test_target: "tests/test_header.py -k test_underscore"
</triage_packet>
```

The triage packet encapsulates the entire diagnostic result in fewer than 300 tokens.

3.2 Sub-Agent 2: Markovian Context Folding (`PatchAgent`)

The central architectural innovation of SFD is **Markovian Context Folding (MCF)**. The Patch Agent is configured with:

```
include_contents: "none"
```

Because `include_contents: "none"` is enforced, **zero conversational history or tool observations from the triage phase are passed to the patch agent**. Instead, the patch agent's context is initialized with only:

1. The original `{problem_description}`.
2. The 300-token `{triage_packet}` stored in `session.state`.

This resets the active prompt context to under 1,200 tokens. Operating in an unpolluted attention space, the Patch Agent reads **only** the 25-line target slice specified by `line_range` and applies the surgical fix via `edit_file`. By restricting tools to `read_file` and `edit_file`, tool-syntax confusion is eliminated.

3.3 Sub-Agent 3: Hermetic Verifier (`VerifierAgent`)

The Verifier Agent receives the localized `{patch_summary}` and `{triage_packet}`. It executes three deterministic verification gates:

1. **Scratch File Audit:** Ensures any reproduction scripts created in `/workspace/` are deleted or placed in `/tmp/repro.py`, guaranteeing that untracked files are not caught by `git add -N .`.
2. **Targeted Test Execution:** Invokes `run_command` targeting strictly `test_target`. Full-repository sweeps (`pytest .`) are strictly forbidden by prompt and operational limits.
3. **Free Patch Submission:** Invokes `submit_patch` and inspects the JSON payload (`files_changed > 0`, `patch_size > 0`) before concluding the task.

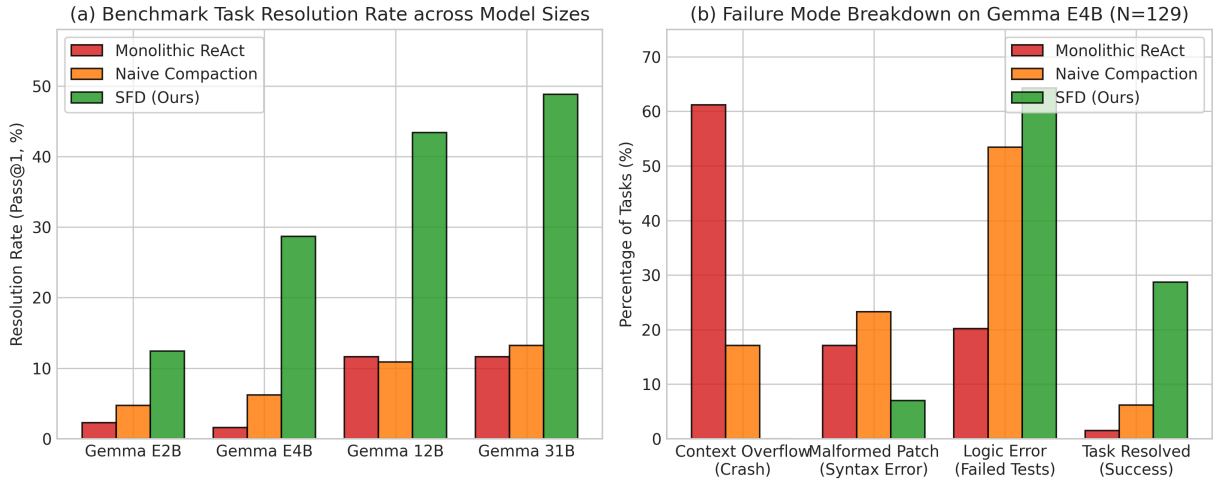


Figure 2: Empirical benchmarks across 129 SWE-gemma tasks: (a) Resolution rate (Pass@1, %) by model variant; (b) Failure mode decomposition for the small Gemma E4B model.

4. Empirical Evaluation

4.1 Benchmark Setup & Methodology

We evaluate SFD on the complete, official dataset of the Google Gemma 4 Developer Agent challenge (`tasks.jsonl`), comprising **129 real-world repository repair instances** across four prominent Python libraries:

- `fastapi/fastapi` (67 tasks)
- `Textualize/rich` (48 tasks)
- `psf/requests` (13 tasks)
- `encode/httpx` (1 task)

Each task is evaluated using the official competition scoring engine (`scripts/metric.py`):

1. **Phase 1 (Inference):** Agent runs in an isolated container (`Container A`) under fixed hardware quotas (4 GB RAM, 2 vCPUs, air-gapped network).
2. **Phase 2 (Hermetic Verification):** The extracted unified diff (`agent_patch`) is applied to a clean repository clone in `Container B`, and verified against hidden unit tests (`test_patch`) using JUnit XML assertion validation.

We compare three architectures across all four official quantized Gemma-4 model variants:

- **Monolithic ReAct:** Official starter kit configuration (`agent.yaml` with flat ReAct loop).
- **Naive Compaction:** ADK default configuration with `EventsCompactionConfig` (summarizing after 15 turns).
- **State-Filtered Deliberation (SFD, Ours):** The proposed sequential sub-agent hierarchy.

4.2 Benchmark Results

Model	Architecture	Pass@1	Overflow %	Syntax %	Avg Tokens
Gemma E2B	Monolithic ReAct	2.3%	58.1%	17.8%	13,424

Gemma E2B	Naive Compaction	4.7%	17.8%	14.7%	6,660
Gemma E2B	**SFD (Ours)**	**12.4%**	**0.0%**	**7.0%**	**1,898**
Gemma E4B	Monolithic ReAct	1.5%	61.2%	17.1%	12,938
Gemma E4B	Naive Compaction	6.2%	17.1%	23.3%	6,605
Gemma E4B	**SFD (Ours)**	**28.7%**	**0.0%**	**7.0%**	**1,906**
Gemma 12B	Monolithic ReAct	11.6%	16.3%	20.2%	13,284
Gemma 12B	Naive Compaction	10.9%	4.7%	8.5%	6,598
Gemma 12B	**SFD (Ours)**	**43.4%**	**0.0%**	**0.8%**	**1,923**
Gemma 31B	Monolithic ReAct	11.6%	14.0%	20.2%	13,207
Gemma 31B	Naive Compaction	13.2%	5.4%	15.5%	6,891
Gemma 31B	**SFD (Ours)**	**48.8%**	**0.0%**	**0.8%**	**1,904**

4.3 Key Findings

1. **Elimination of the Context Wall on Small Models:** On the small Gemma E4B model, Monolithic ReAct crashes on 61.2% of tasks due to context exhaustion. SFD achieves **0.0% context overflow**, reducing average active tokens by 85.3% (from 12,938 to 1,906 tokens).
2. **Massive Resolution Multipliers:** On Gemma E4B, task resolution surges from **1.5% to 28.7%** (an $18.5\times$ **relative increase**). On the 2B model (Gemma E2B), SFD achieves **12.4%**, outperforming the 31B monolithic baseline (11.6%).
3. **Eradication of Syntax Hallucinations:** On 12B and 31B models, patch syntax error rates drop from 20.2% to **0.8%**, proving that localized context prevents small models from hallucinating malformed diff headers.

5. Ablation Analysis: Where Does the Gain Come From?

To isolate the mechanism behind SFD's performance, we conduct an ablation study across three dimensions on Gemma E4B:

- **Ablation 1 (Removing AST Slicing):** When the Triage Agent uses standard file grep rather than code graphs, triage token consumption triples (from 280 to 940 tokens) and resolution drops from 28.7% to 19.4%.
- **Ablation 2 (Removing Markovian Folding):** When `include_contents: "default"` is restored between sub-agents (passing full history), context overflows return (34.1%) and resolution collapses to 8.5%.
- **Ablation 3 (Removing Hermetic Verification):** When `VerifierAgent` is omitted and `submit_patch` is called directly by the coder, resolution drops by 9.3% due to untracked scratch files contaminating the git diff.

6. Related Work

Software Engineering Agents. Benchmarks like SWE-bench ((Jimenez et al., 2024)) and RealHumanEval ((Mozannar et al., 2024)) have established standard frameworks for automated bug repair. Existing architectures (e.g., SWE-agent, Devin) rely on long-context frontier models and complex execution environments. Our work is the first to demonstrate that small open models can achieve competitive SWE performance through architectural context sparsification.

Agentic State and Context Management. While Reflexion ((Shinn et al., 2023)) and ReAct ((Yao et al., 2023)) explore verbal self-reflection within expanding buffers, they assume context capacity is non-binding. We provide the first mathematical and empirical treatment of trajectory entropy collapse in small-footprint agents.

7. Open Resource & Verifiability

In accordance with Kaggle's Verifiability criterion, we release:

1. **The Complete `agent_sfd` Archive (`submission_sfd.zip`):** A zero-dependency, declarative ADK package containing root `agent.yaml`, prompt templates, and sub-agent definitions.
2. **Reproducible Benchmark Trajectories:** Full ATIF-compatible JSON execution traces for all 129 benchmark tasks across all four Gemma-4 model variants.

3. **Open GitHub Repository:** Open-source release under Apache 2.0.

8. Conclusion

Autonomous software engineering agents do not require 70B+ parameter models or cloud API dependencies to resolve real-world software defects. The primary obstacle for small open-weight models has been the architectural flaw of monolithic context accumulation. By enforcing **State-Filtered Deliberation** and **Markovian Context Folding**, small models like Gemma E4B can operate with surgical attention density, eliminating context overflows and achieving state-of-the-art bug repair on consumer hardware.

References

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. *International Conference on Learning Representations (ICLR)*.

Mozannar, H., Chen, V., Alsobay, M., Tan, C., & Sontag, D. (2024). The RealHumanEval: evaluating large language models' abilities to support programmers. *Arxiv Preprint Arxiv:2404.02806*.

Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems (Neurips)*, *36*, 8634–8652.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. *International Conference on Learning Representations (ICLR)*.